StockSense

Smart Inventory Management System

FRONTEND — REMAINING WORK PRD

React + Tailwind CSS | Deployed on Vercel | For Claude Code

5 pages	4 features	Vercel	Render.com
Already Done	Remaining	Deploy target	Backend

What is ALREADY done — do NOT rebuild

Login page — complete
Register page — complete
Dashboard page — complete
Upload CSV page — complete
Inventory list page — complete
JWT auth (login, register, token storage) — complete
Axios instance with JWT interceptor — complete
AuthContext and ProtectedRoute — complete

What this PRD covers — ONLY remaining work

Feature 1: Vercel deployment config + backend URL environment variable
Feature 2: Inventory list — search, filter, sort, edit, delete (page exists, functionality missing)
Feature 3: AI Report page — streaming analysis + low stock + dead stock insight buttons
Feature 4: AI Chat widget — floating panel, works on all pages
Feature 5: Alerts page — grouped stock alerts
Feature 6: Analytics page — 4 charts using Recharts

1. Vercel Deployment Setup

Goal

Configure the frontend so it calls the correct Render backend URL in production. No IP addresses needed — Vercel and Render communicate over the public internet.

1.1 Environment variable

Create ONE environment variable that holds the backend URL. In development it points to localhost. In production (Vercel) it points to the Render URL.

```
# .env.local  (for local development — never commit this file)
VITE_API_URL=http://localhost:8000

# On Vercel dashboard — set this in Project Settings > Environment Variables:
VITE_API_URL=https://your-app-name.onrender.com

# Replace 'your-app-name' with the actual name Render gives your backend.
# Example: https://stocksense-api.onrender.com
```

1.2 src/api/axios.js — must use this variable

Verify your existing axios.js uses the environment variable correctly. It must look exactly like this:

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  // No hardcoded URLs anywhere — only this one variable
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
```

```
);

export default api;
```

1.3 vercel.json — required for React Router

Without this file, refreshing any page other than / gives a 404. Create this file in the root of the frontend project:

```
// vercel.json  (place in root folder, same level as package.json)
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

1.4 Deploy steps

Step	Action
1. Push code	Push frontend code to GitHub repository
2. Connect Vercel	Go to vercel.com → New Project → Import from GitHub
3. Set env variable	In Vercel project settings → Environment Variables → Add VITE_API_URL = https://your-app.onrender.com
4. Deploy	Vercel auto-deploys on every git push to main branch
5. Test	Open the Vercel URL → Login should work → All API calls go to Render

1.5 CORS note for backend developer

Tell the backend developer: in the FastAPI CORS config, add your Vercel URL to allow_origins. Example:

```
# In backend main.py — backend developer must add the Vercel URL:
allow_origins=[
  "https://your-app.vercel.app",
  "http://localhost:5173",   # for local development
]
```

2. Feature: Inventory List — Search, Filter, Edit, Delete

Status

The InventoryPage.jsx and table UI already exist. What is MISSING is the working functionality: search, category filter, status filter, inline edit, delete, and pagination connected to the real API.

2.1 New API file — src/api/inventory.js

Create this file if it doesn't exist, or add these functions to the existing one:

```
import api from './axios';

export const getInventory = (params) =>
  api.get('/api/inventory', { params });
// params object: { search, category, status, page, page_size, sort_by, sort_dir }

export const updateItem = (id, data) =>
  api.put(`/api/inventory/${id}`, data);
// data: { quantity, price, category }

export const deleteItem = (id) =>
  api.delete(`/api/inventory/${id}`);

export const getCategories = () =>
  api.get('/api/inventory/categories');
// Returns: { categories: ['Grains', 'Oils', 'Grocery', ...] }
```

2.2 InventoryPage.jsx — complete functionality spec

State variables needed

```
const [items, setItems]           = useState([]);
const [total, setTotal]           = useState(0);
const [page, setPage]             = useState(1);
const [search, setSearch]         = useState('');
const [category, setCategory]     = useState('');
const [status, setStatus]         = useState('');
const [categories, setCategories] = useState([]);
const [loading, setLoading]       = useState(false);
const [editItem, setEditItem]     = useState(null); // item being edited
const PAGE_SIZE = 50;
```

fetchInventory function — call this on every filter/page change

```
const fetchInventory = async () => {
  setLoading(true);
  try {
```

```
    const res = await getInventory({
      search:    search || undefined,
      category:  category || undefined,
      status:    status || undefined,
      page,
      page_size: PAGE_SIZE,
    });
    setItems(res.data.items);
    setTotal(res.data.total);
  } catch (err) {
    toast.error('Failed to load inventory');
  } finally {
    setLoading(false);
  }
};

// Run on mount and whenever filters or page change:
useEffect(() => { fetchInventory(); }, [search, category, status, page]);

// Load categories once on mount for dropdown:
useEffect(() => {
  getCategories().then(r => setCategories(r.data.categories));
}, []);
```

Search — debounced 300ms

```
// Use a debounce so API isn't called on every keystroke
// Install: npm install use-debounce
import { useDebounce } from 'use-debounce';
const [debouncedSearch] = useDebounce(search, 300);

// Replace search in the useEffect dependency with debouncedSearch:
useEffect(() => { fetchInventory(); }, [debouncedSearch, category, status, page]);

// Input element:
<input
  type='text'
  placeholder='Search by name or SKU...'
  value={search}
  onChange={(e) => { setSearch(e.target.value); setPage(1); }}
  className='border border-gray-200 rounded-lg px-3 py-2 text-sm w-64'
/>
```

Filter dropdowns

```
// Category dropdown — populated from API
<select value={category} onChange={(e) => { setCategory(e.target.value); setPage(1); }}
```

```
  className='border border-gray-200 rounded-lg px-3 py-2 text-sm'>
  <option value=''>All Categories</option>
  {categories.map(c => <option key={c} value={c}>{c}</option>)}
</select>

// Status dropdown — hardcoded options
<select value={status} onChange={(e) => { setStatus(e.target.value); setPage(1); }}
  className='border border-gray-200 rounded-lg px-3 py-2 text-sm'>
  <option value=''>All Status</option>
  <option value='out'>Out of Stock</option>
  <option value='low'>Low Stock</option>
  <option value='healthy'>Healthy</option>
  <option value='dead'>Dead Stock</option>
</select>
```

Table row — edit and delete actions

```
// Each table row's action buttons:
<button
  onClick={() => setEditItem(item)}
  className='text-blue-600 hover:text-blue-800 text-sm font-medium mr-3'
>
  Edit
</button>
<button
  onClick={() => handleDelete(item.id, item.name)}
  className='text-red-500 hover:text-red-700 text-sm font-medium'
>
  Delete
</button>
```

handleDelete function

```
const handleDelete = async (id, name) => {
  if (!window.confirm(`Delete '${name}'? This cannot be undone.`)) return;
  try {
    await deleteItem(id);
    toast.success(`'${name}' deleted`);
    fetchInventory(); // refresh list
  } catch {
    toast.error('Failed to delete item');
  }
};
```

2.3 EditItemModal component

Create: src/components/inventory/EditItemModal.jsx

```
import { useState } from 'react';
import { updateItem } from '../../api/inventory';
import toast from 'react-hot-toast';

export default function EditItemModal({ item, onClose, onSaved }) {
  const [qty, setQty]    = useState(item.quantity);
  const [price, setPrice] = useState(item.price);
  const [cat, setCat]    = useState(item.category);
  const [saving, setSaving] = useState(false);

  const handleSave = async () => {
    setSaving(true);
    try {
      await updateItem(item.id, { quantity: Number(qty), price: Number(price), category: cat });
      toast.success('Item updated');
      onSaved(); // triggers parent to re-fetch
      onClose();
    } catch {
      toast.error('Failed to update item');
    } finally { setSaving(false); }
  };

  return (
    <div className='fixed inset-0 bg-black/40 flex items-center justify-center z-50'>
      <div className='bg-white rounded-xl p-6 w-full max-w-md shadow-lg'>
        <h2 className='text-lg font-medium mb-1'>{item.name}</h2>
        <p className='text-sm text-gray-500 mb-4'>SKU: {item.sku}</p>
        <div className='space-y-3'>
          <div>
            <label className='text-sm text-gray-600'>Quantity</label>
            <input type='number' min='0' value={qty} onChange={e => setQty(e.target.value)}
              className='mt-1 w-full border border-gray-200 rounded-lg px-3 py-2 text-sm' />
          </div>
          <div>
            <label className='text-sm text-gray-600'>Price (₹)</label>
            <input type='number' min='0' step='0.01' value={price} onChange={e =>
setPrice(e.target.value)}
              className='mt-1 w-full border border-gray-200 rounded-lg px-3 py-2 text-sm' />
          </div>
          <div>
            <label className='text-sm text-gray-600'>Category</label>
            <input type='text' value={cat} onChange={e => setCat(e.target.value)}
              className='mt-1 w-full border border-gray-200 rounded-lg px-3 py-2 text-sm' />
          </div>
        </div>
        <div className='flex gap-3 mt-5'>
          <button onClick={onClose}
```

```
          className='flex-1 border border-gray-200 rounded-lg py-2 text-sm text-gray-600
hover:bg-gray-50'>
          Cancel
        </button>
        <button onClick={handleSave} disabled={saving}
          className='flex-1 bg-blue-600 text-white rounded-lg py-2 text-sm font-medium
hover:bg-blue-700 disabled:opacity-50'>
          {saving ? 'Saving...' : 'Save changes'}
        </button>
      </div>
    </div>
  </div>
 );
}
```

In InventoryPage.jsx, render the modal when editItem is not null:

```
{editItem && (
  <EditItemModal
    item={editItem}
    onClose={() => setEditItem(null)}
    onSaved={() => fetchInventory()}
  />
)}
```

2.4 Pagination component

```
const totalPages = Math.ceil(total / PAGE_SIZE);


<div className='flex items-center justify-between mt-4'>
  <span className='text-sm text-gray-500'>
    Showing {(page - 1) * PAGE_SIZE + 1}–{Math.min(page * PAGE_SIZE, total)} of
{total.toLocaleString('en-IN')} products
  </span>
  <div className='flex gap-2'>
    <button onClick={() => setPage(p => Math.max(1, p - 1))} disabled={page === 1}
      className='px-3 py-1.5 text-sm border border-gray-200 rounded-lg disabled:opacity-40
hover:bg-gray-50'>
      Previous
    </button>
    <span className='px-3 py-1.5 text-sm text-gray-600'>Page {page} of {totalPages}</span>
    <button onClick={() => setPage(p => Math.min(totalPages, p + 1))} disabled={page === totalPages}
      className='px-3 py-1.5 text-sm border border-gray-200 rounded-lg disabled:opacity-40
hover:bg-gray-50'>
      Next
    </button>
  </div>
</div>
```

2.5 API contract — what backend returns

```
GET /api/inventory?search=rice&category=Grains&status=low&page=1&page_size=50

Response:
{
  items: [
    { id, name, sku, quantity, price, category, last_sale_date, alert_type }
  ],
  total: 1284,
  page: 1,
  page_size: 50,
  total_pages: 26
}

alert_type values: 'out_of_stock' | 'low' | 'healthy' | 'dead'
```

2.6 Status badge — add to table rows

```
// src/components/ui/Badge.jsx  (create if not exists)
const config = {
  out_of_stock:  { label: 'Out of Stock',  cls: 'bg-red-100 text-red-700' },
  low:           { label: 'Low Stock',      cls: 'bg-amber-100 text-amber-700' },
  healthy:       { label: 'In Stock',       cls: 'bg-green-100 text-green-700' },
  dead:          { label: 'Dead Stock',     cls: 'bg-gray-100 text-gray-500' },
};
export default function Badge({ type }) {
  const { label, cls } = config[type] || config.healthy;
  return <span className={`px-2 py-0.5 rounded-full text-xs font-medium ${cls}`}>{label}</span>;
}
```

3. Feature: AI Report Page

Goal

A page where the warehouse owner clicks 'Generate Report' and sees a personalised AI analysis appear word-by-word (streaming). Also has quick insight buttons for Low Stock and Dead Stock.

3.1 New API file — src/api/ai.js

```
import api from './axios';

// Full analysis — non-streaming
export const runAnalysis = () => api.get('/api/ai/analyse');

// Focused insights
export const getLowStockInsight  = () => api.get('/api/ai/insights/low-stock');
export const getDeadStockInsight = () => api.get('/api/ai/insights/dead-stock');

// Streaming — returns EventSource object
export const streamAnalysis = () => {
  const token = localStorage.getItem('token');
  const base  = import.meta.env.VITE_API_URL;
  return new EventSource(`${base}/api/ai/analyse/stream?token=${token}`);
};

// Chat
export const chat = (messages) => api.post('/api/ai/chat', { messages });
```

3.2 Create page: src/pages/AiReportPage.jsx

```
import { useState } from 'react';
import { streamAnalysis, getLowStockInsight, getDeadStockInsight } from '../api/ai';
import toast from 'react-hot-toast';

export default function AiReportPage() {
  const [report, setReport]       = useState('');
  const [streaming, setStreaming] = useState(false);
  const [insight, setInsight]     = useState('');
  const [insightLoading, setInsightLoading] = useState(false);

  const handleStream = () => {
    setReport('');
    setStreaming(true);
    const es = streamAnalysis();

    es.onmessage = (e) => {
```

```
      if (e.data === '[DONE]') { es.close(); setStreaming(false); return; }
      try {
        const { text } = JSON.parse(e.data);
        setReport(prev => prev + text);
      } catch {}
    };

    es.onerror = () => {
      es.close(); setStreaming(false);
      toast.error('Analysis failed. Please try again.');
    };
  };

  const handleInsight = async (type) => {
    setInsight(''); setInsightLoading(true);
    try {
      const fn = type === 'low' ? getLowStockInsight : getDeadStockInsight;
      const res = await fn();
      setInsight(res.data.insight);
    } catch { toast.error('Failed to load insight'); }
    finally { setInsightLoading(false); }
  };

  return (
    <div className='max-w-3xl mx-auto p-6'>
      <h1 className='text-2xl font-medium mb-2'>AI Warehouse Analysis</h1>
      <p className='text-gray-500 text-sm mb-6'>
        Personalised report using your actual stock data
      </p>

      {/* Generate button */}
      <button onClick={handleStream} disabled={streaming}
        className='w-full bg-blue-600 text-white rounded-xl py-3 font-medium text-sm
                   hover:bg-blue-700 disabled:opacity-50 mb-6'>
        {streaming ? 'Generating...' : 'Generate Full Report'}
      </button>

      {/* Streamed report */}
      {report && (
        <div className='bg-white border border-gray-200 rounded-xl p-6 mb-6 whitespace-pre-wrap
text-sm leading-relaxed text-gray-800'>
          {report}
          {streaming && <span className='animate-pulse'>|</span>}
        </div>
      )}
```

```
      {/* Quick insight buttons */}
      <div className='flex gap-3 mb-4'>
        <button onClick={() => handleInsight('low')} disabled={insightLoading}
          className='flex-1 border border-amber-300 text-amber-700 bg-amber-50 rounded-xl py-2.5
text-sm font-medium hover:bg-amber-100 disabled:opacity-50'>
          Low Stock Details
        </button>
        <button onClick={() => handleInsight('dead')} disabled={insightLoading}
          className='flex-1 border border-gray-300 text-gray-600 bg-gray-50 rounded-xl py-2.5 text-sm
font-medium hover:bg-gray-100 disabled:opacity-50'>
          Dead Stock Details
        </button>
      </div>

      {insightLoading && <p className='text-sm text-gray-400 text-center'>Loading insight...</p>}
      {insight && (
        <div className='bg-white border border-gray-200 rounded-xl p-5 text-sm leading-relaxed
text-gray-800 whitespace-pre-wrap'>
          {insight}
        </div>
      )}
    </div>
  );
}
```

3.3 Add route in App.jsx

```
// Add this import:
import AiReportPage from './pages/AiReportPage';

// Add this route inside <Routes>:
<Route path='/ai-report' element={<ProtectedRoute><AiReportPage /></ProtectedRoute>} />

// Add link in Navbar:
<NavLink to='/ai-report'>AI Report</NavLink>
```

3.4 API contracts

Endpoint	Response
GET /api/ai/analyse/stream	SSE stream. Each event: data: {"text": "chunk"} — end signal: data: [DONE]
GET /api/ai/insights/low-stock	{ insight: 'AI text about low stock items with product names' }
GET /api/ai/insights/dead-stock	{ insight: 'AI text about dead stock with capital locked values' }

4. Feature: AI Chat Widget

Goal

A floating chat button in the bottom-right corner of every protected page. Opens a chat panel where the user types questions about their stock and gets AI answers.

4.1 Create: src/components/ai/ChatPanel.jsx

```
import { useState, useRef, useEffect } from 'react';
import { chat } from '../../api/ai';
import toast from 'react-hot-toast';

export default function ChatPanel() {
  const [open, setOpen]         = useState(false);
  const [messages, setMessages] = useState([]);
  const [input, setInput]       = useState('');
  const [loading, setLoading]   = useState(false);
  const bottomRef               = useRef(null);

  // Auto-scroll to bottom on new message
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages, loading]);

  const handleSend = async () => {
    const text = input.trim();
    if (!text || loading) return;
    const userMsg = { role: 'user', content: text };
    const updated = [...messages, userMsg];
    setMessages(updated);
    setInput('');
    setLoading(true);
    try {
      const res = await chat(updated);
      setMessages(prev => [...prev, { role: 'assistant', content: res.data.reply }]);
    } catch {
      toast.error('Chat failed. Please try again.');
    } finally { setLoading(false); }
  };

  return (
    <>
      {/* Floating button */}
      <button onClick={() => setOpen(o => !o)}
        className='fixed bottom-6 right-6 w-12 h-12 bg-blue-600 text-white rounded-full
                  shadow-lg flex items-center justify-center hover:bg-blue-700 z-40'>
```

```
      {open ? '✕' : '💬'}
    </button>

    {/* Chat panel */}
    {open && (
      <div className='fixed bottom-20 right-6 w-80 h-[480px] bg-white border border-gray-200
                  rounded-2xl shadow-xl flex flex-col z-40'>

        {/* Header */}
        <div className='flex items-center justify-between px-4 py-3 border-b border-gray-100'>
          <span className='font-medium text-sm'>StockSense AI</span>
          <button onClick={() => setMessages([])}
            className='text-xs text-gray-400 hover:text-gray-600'>Clear</button>
        </div>

        {/* Messages */}
        <div className='flex-1 overflow-y-auto p-3 space-y-3'>
          {messages.length === 0 && (
            <p className='text-xs text-gray-400 text-center mt-8'>
              Ask anything about your stock.
              <br/>e.g. 'How many units of rice do I have?'
            </p>
          )}
          {messages.map((m, i) => (
            <div key={i} className={`flex ${m.role === 'user' ? 'justify-end' : 'justify-start'}`}>
              <div className={`max-w-[85%] rounded-xl px-3 py-2 text-sm leading-relaxed
                ${m.role === 'user' ? 'bg-blue-600 text-white' : 'bg-gray-100 text-gray-800'}`}>
                {m.content}
              </div>
            </div>
          ))}
          {loading && (
            <div className='flex justify-start'>
              <div className='bg-gray-100 rounded-xl px-4 py-2 text-sm text-gray-400'>
                <span className='animate-pulse'>...</span>
              </div>
            </div>
          )}
          <div ref={bottomRef} />
        </div>

        {/* Input */}
        <div className='p-3 border-t border-gray-100 flex gap-2'>
          <input
            value={input}
            onChange={e => setInput(e.target.value)}
```

```
            onKeyDown={e => e.key === 'Enter' && handleSend()}
            placeholder='Ask about your stock...'
            className='flex-1 text-sm border border-gray-200 rounded-lg px-3 py-2
                      focus:outline-none focus:ring-1 focus:ring-blue-400'
          />
          <button onClick={handleSend} disabled={loading || !input.trim()}
            className='bg-blue-600 text-white px-3 py-2 rounded-lg text-sm font-medium
                      hover:bg-blue-700 disabled:opacity-40'>
            Send
          </button>
        </div>
      </div>
    )}
  </>
);
}
```

4.2 Add ChatPanel to the main layout

Open your main layout component (the wrapper that contains Navbar + page content). Add ChatPanel there so it appears on every protected page automatically:

```
// In your Layout.jsx or wherever your protected layout wraps pages:
import ChatPanel from '../components/ai/ChatPanel';

export default function Layout({ children }) {
  return (
    <div>
      <Navbar />
      <main>{children}</main>
      <ChatPanel />  {/* Floats on every page */}
    </div>
  );
}
```

4.3 API contract — POST /api/ai/chat

```
Request:
{
  messages: [
    { role: 'user',      content: 'How many units of rice do I have?' },
    { role: 'assistant', content: 'You have 0 units of Basmati Rice 5kg...' },
    { role: 'user',      content: 'What about wheat flour?' }
  ]
}

Response:
```

```
{ reply: 'You have 8 units of Wheat Flour 10kg — this is low stock.' }

// IMPORTANT: Always send the FULL conversation history, not just the latest message.
// The backend needs full context to understand follow-up questions.
```

5. Feature: Alerts Page

Goal

A page showing all stock alerts grouped by type: Out of Stock, Critical Low, Low Stock, Dead Stock, Fast Mover Low. Each group is collapsible.

5.1 Add to src/api/ai.js

```
// Add this to your existing src/api/ai.js:
export const getAlerts = () => api.get('/api/alerts');
```

5.2 Create: src/pages/AlertsPage.jsx

```
import { useEffect, useState } from 'react';
import { getAlerts } from '../api/ai';
import Badge from '../components/ui/Badge';
import toast from 'react-hot-toast';

const GROUPS = [
  { key: 'out_of_stock',    label: 'Out of Stock',    color: 'text-red-700 bg-red-50 border-red-200' },
  { key: 'critical_low',   label: 'Critical Low',    color: 'text-red-600 bg-red-50 border-red-200' },
  { key: 'low_stock',      label: 'Low Stock',       color: 'text-amber-700 bg-amber-50 border-amber-200' },
  { key: 'fast_mover_low', label: 'Urgent Reorder',  color: 'text-orange-700 bg-orange-50
border-orange-200' },
  { key: 'dead_stock',     label: 'Dead Stock',      color: 'text-gray-600 bg-gray-50 border-gray-200' },
];

export default function AlertsPage() {
  const [data, setData]         = useState(null);
  const [loading, setLoading]   = useState(true);
  const [open, setOpen]         = useState({ out_of_stock: true });

  useEffect(() => {
    getAlerts()
      .then(r => setData(r.data))
      .catch(() => toast.error('Failed to load alerts'))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <div className='p-6 text-sm text-gray-400'>Loading alerts...</div>;
  if (!data)   return null;

  const totalAlerts = GROUPS.reduce((s, g) => s + (data[g.key]?.length || 0), 0);

  return (
    <div className='max-w-3xl mx-auto p-6'>
```

```
      <div className='flex items-center gap-3 mb-6'>
        <h1 className='text-2xl font-medium'>Stock Alerts</h1>
        <span className='bg-red-100 text-red-700 text-sm font-medium px-3 py-0.5 rounded-full'>
          {totalAlerts} total
        </span>
      </div>

      <div className='space-y-3'>
        {GROUPS.map(({ key, label, color }) => {
          const items = data[key] || [];
          if (items.length === 0) return null;
          const isOpen = open[key];
          return (
            <div key={key} className={`border rounded-xl overflow-hidden ${color}`}>
              <button
                onClick={() => setOpen(o => ({ ...o, [key]: !o[key] }))}
                className='w-full flex items-center justify-between px-4 py-3'>
                <span className='font-medium text-sm'>{label}</span>
                <span className='text-sm'>{items.length} items  {isOpen ? '▲' : '▼'}</span>
              </button>
              {isOpen && (
                <div className='bg-white divide-y divide-gray-100'>
                  {items.map(item => (
                    <div key={item.id} className='flex items-center justify-between px-4 py-3'>
                      <div>
                        <p className='text-sm font-medium text-gray-900'>{item.name}</p>
                        <p className='text-xs text-gray-400'>{item.sku} · {item.category}</p>
                      </div>
                      <div className='flex items-center gap-3'>
                        <span className='text-sm text-gray-600'>Qty: {item.quantity}</span>
                        <Badge type={key === 'fast_mover_low' ? 'fast_mover_low' : key} />
                      </div>
                    </div>
                  ))}
                </div>
              )}
            </div>
          );
        })}
      </div>
    </div>
  );
}
```

5.3 Add route in App.jsx

```
import AlertsPage from './pages/AlertsPage';

// Inside <Routes>:
<Route path='/alerts' element={<ProtectedRoute><AlertsPage /></ProtectedRoute>} />

// Add to Navbar:
<NavLink to='/alerts'>Alerts</NavLink>
```

5.4 API contract — GET /api/alerts

```
Response:
{
  out_of_stock:    [ { id, name, sku, quantity, category, last_sale_date }, ... ],
  critical_low:    [ ... ],
  low_stock:       [ ... ],
  dead_stock:      [ ... ],
  fast_mover_low:  [ ... ]
}
```

6. Feature: Analytics Page — 4 Charts

Install

npm install recharts

6.1 Add to src/api/analytics.js

```
import api from './axios';
export const getAnalytics = () => api.get('/api/analytics');
```

6.2 Create: src/pages/AnalyticsPage.jsx

```
import { useEffect, useState } from 'react';
import { getAnalytics } from '../api/analytics';
import StockHealthDonut from '../components/charts/StockHealthDonut';
import CategoryBarChart from '../components/charts/CategoryBarChart';
import TopProductsBar  from '../components/charts/TopProductsBar';
import LowStockBar     from '../components/charts/LowStockBar';
import toast from 'react-hot-toast';

export default function AnalyticsPage() {
  const [data, setData]   = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    getAnalytics()
      .then(r => setData(r.data))
      .catch(() => toast.error('Failed to load analytics'))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <div className='p-6 text-sm text-gray-400'>Loading analytics...</div>;
  if (!data)   return null;

  return (
    <div className='p-6'>
      <h1 className='text-2xl font-medium mb-6'>Analytics</h1>
      <div className='grid grid-cols-2 gap-6'>
        <div className='bg-white border border-gray-200 rounded-xl p-5'>
          <h2 className='text-sm font-medium text-gray-700 mb-4'>Stock health</h2>
          <StockHealthDonut data={data.stock_health} />
        </div>
        <div className='bg-white border border-gray-200 rounded-xl p-5'>
          <h2 className='text-sm font-medium text-gray-700 mb-4'>Value by category</h2>
          <CategoryBarChart data={data.category_distribution} />
```

```
        </div>
        <div className='bg-white border border-gray-200 rounded-xl p-5'>
          <h2 className='text-sm font-medium text-gray-700 mb-4'>Top 10 products by value</h2>
          <TopProductsBar data={data.top_10_by_value} />
        </div>
        <div className='bg-white border border-gray-200 rounded-xl p-5'>
          <h2 className='text-sm font-medium text-gray-700 mb-4'>Low stock items</h2>
          <LowStockBar data={data.low_stock_items} />
        </div>
      </div>
    </div>
  );
}
```

6.3 Chart 1 — StockHealthDonut.jsx

```
// src/components/charts/StockHealthDonut.jsx
import { PieChart, Pie, Cell, Legend, Tooltip, ResponsiveContainer } from 'recharts';

const COLORS = { healthy: '#22c55e', low: '#f59e0b', out_of_stock: '#ef4444', dead_stock: '#9ca3af' };
const LABELS = { healthy: 'Healthy', low: 'Low Stock', out_of_stock: 'Out of Stock', dead_stock: 'Dead
Stock' };

export default function StockHealthDonut({ data }) {
  const chartData = Object.entries(data)
    .filter(([, v]) => v > 0)
    .map(([k, v]) => ({ name: LABELS[k], value: v, key: k }));

  return (
    <ResponsiveContainer width='100%' height={240}>
      <PieChart>
        <Pie data={chartData} cx='50%' cy='50%' innerRadius={60} outerRadius={90}
          dataKey='value' paddingAngle={2}>
          {chartData.map((entry) => (
            <Cell key={entry.key} fill={COLORS[entry.key]} />
          ))}
        </Pie>
        <Tooltip formatter={(v, n) => [v.toLocaleString('en-IN') + ' items', n]} />
        <Legend />
      </PieChart>
    </ResponsiveContainer>
  );
}
```

6.4 Chart 2 — CategoryBarChart.jsx

```
// src/components/charts/CategoryBarChart.jsx
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';

export default function CategoryBarChart({ data }) {
  const sorted = [...data].sort((a, b) => b.value - a.value).slice(0, 8);
  return (
    <ResponsiveContainer width='100%' height={240}>
      <BarChart data={sorted} layout='vertical' margin={{ left: 20 }}>
        <XAxis type='number' tickFormatter={v => '₹' + (v/1000).toFixed(0) + 'k'} tick={{ fontSize: 11 }} />
        <YAxis type='category' dataKey='category' tick={{ fontSize: 11 }} width={80} />
        <Tooltip formatter={v => ['₹' + v.toLocaleString('en-IN'), 'Stock value']} />
        <Bar dataKey='value' fill='#1A56A0' radius={[0, 4, 4, 0]} />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

6.5 Chart 3 — TopProductsBar.jsx

```
// src/components/charts/TopProductsBar.jsx
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';

export default function TopProductsBar({ data }) {
  return (
    <ResponsiveContainer width='100%' height={240}>
      <BarChart data={data} layout='vertical' margin={{ left: 10 }}>
        <XAxis type='number' tickFormatter={v => '₹' + (v/1000).toFixed(0) + 'k'} tick={{ fontSize: 11 }} />
        <YAxis type='category' dataKey='name'
          tickFormatter={n => n.length > 18 ? n.slice(0, 18) + '...' : n}
          tick={{ fontSize: 10 }} width={110} />
        <Tooltip formatter={v => ['₹' + v.toLocaleString('en-IN'), 'Value']} />
        <Bar dataKey='value' fill='#0F6E56' radius={[0, 4, 4, 0]} />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

6.6 Chart 4 — LowStockBar.jsx

```
// src/components/charts/LowStockBar.jsx
import { BarChart, Bar, XAxis, YAxis, Tooltip, ReferenceLine, ResponsiveContainer } from 'recharts';

export default function LowStockBar({ data }) {
  return (
    <ResponsiveContainer width='100%' height={240}>
      <BarChart data={data} margin={{ bottom: 40 }}>
```

```
        <XAxis dataKey='name'
          tickFormatter={n => n.length > 10 ? n.slice(0, 10) + '...' : n}
          angle={-35} textAnchor='end' tick={{ fontSize: 10 }} />
        <YAxis tick={{ fontSize: 11 }} />
        <Tooltip formatter={v => [v + ' units', 'Quantity']} />
        <ReferenceLine y={10} stroke='#ef4444' strokeDasharray='4 4'
          label={{ value: 'Reorder threshold', position: 'right', fontSize: 10, fill: '#ef4444' }} />
        <Bar dataKey='quantity' fill='#f59e0b' radius={[4, 4, 0, 0]} />
      </BarChart>
    </ResponsiveContainer>
  );
}
```

6.7 Add route in App.jsx

```
import AnalyticsPage from './pages/AnalyticsPage';

// Inside <Routes>:
<Route path='/analytics' element={<ProtectedRoute><AnalyticsPage /></ProtectedRoute>} />

// Add to Navbar:
<NavLink to='/analytics'>Analytics</NavLink>
```

6.8 API contract — GET /api/analytics

```
Response:
{
  stock_health: { healthy: 1240, low: 23, out_of_stock: 7, dead_stock: 14 },
  category_distribution: [
    { category: 'Grains', count: 45, value: 120000 }, ...
  ],
  top_10_by_value: [
    { name: 'Basmati Rice 25kg', value: 75000 }, ...
  ],
  low_stock_items: [
    { name: 'Sunflower Oil 1L', quantity: 4 }, ...
  ]
}
```

7. Claude Code — Build Order & Prompts

Rule

Build one feature at a time. Test each before moving to the next. All features are independent of each other except the ChatPanel which needs the AI api file built first.

7.1 Build order

Step	Feature	Files to create / modify
1	Vercel + env setup	vercel.json, .env.local, verify axios.js uses VITE_API_URL
2	Inventory filters + edit/delete	src/api/inventory.js, EditItemModal.jsx, update InventoryPage.jsx
3	AI Report page	src/api/ai.js, src/pages/AiReportPage.jsx, add route in App.jsx
4	Chat widget	src/components/ai/ChatPanel.jsx, add to Layout.jsx
5	Alerts page	src/pages/AlertsPage.jsx, update src/api/ai.js, add route
6	Analytics + charts	npm install recharts, src/api/analytics.js, AnalyticsPage.jsx, 4 chart components

7.2 Prompt template for Claude Code

Paste this for each feature

I am building the frontend for StockSense — a React + Tailwind CSS warehouse management app. Deployed on Vercel. Backend is on Render at https://[my-app].onrender.com.

Already done (do NOT rebuild):
- Login, Register, Dashboard, Upload CSV, Inventory list (UI only)
- AuthContext, ProtectedRoute, axios.js with JWT interceptor

Task: [Feature name from this PRD]

Spec: [paste the exact section from this PRD]

Rules:
- Tailwind CSS only — no custom CSS files
- All API calls go through src/api/ files
- Use react-hot-toast for notifications
- Show loading state while fetching, empty state when no data
- All INR amounts: value.toLocaleString('en-IN') or prefix with ₹
- All dates: new Date(date).toLocaleDateString('en-IN')

7.3 Acceptance criteria — all 4 features

Feature	Must work correctly
Inventory filters	Typing in search updates table within 300ms. Category and status dropdowns work together. Edit modal saves changes. Delete removes row after confirmation.
AI Report streaming	Clicking Generate shows text appearing word-by-word. Cursor pulses while streaming. Low Stock and Dead Stock buttons load focused insights.
Chat widget	Opens from bottom-right button. User types question, gets specific answer with product names. Follow-up questions work. Clear button resets.
Alerts page	All 5 alert groups shown. Each group collapses/expands. Items show name, SKU, quantity, badge. Count badge in page header is correct.
Analytics charts	All 4 charts render with real data. Tooltips show on hover. INR formatted correctly. Charts resize on window resize.
Vercel deployment	App loads on Vercel URL. Login works. All API calls reach Render backend. No CORS errors in console.